

C# - Phần 9.5: Predicate<T>

Trọng tâm: hiểu Predicate thông qua List<T>.Find(), FindAll(), Exists(), RemoveAll()

Mục tiêu bài học: Không học Predicate quá sâu. Chỉ cần hiểu bản chất của Predicate<T>, vì sao List<T> dùng nó, và cách nhận ra Predicate ngay cả khi từ "Predicate" không xuất hiện ở dòng gọi method.

1. Predicate<T> là gì?

Predicate<T> là một Generic Delegate có sẵn trong .NET. Nó luôn nhận đúng một giá trị kiểu T và trả về bool.

```
Predicate<T>  
≈  
bool Method(T value)
```

Ví dụ:

```
Predicate<int>          // nhận int, trả bool  
Predicate<string>      // nhận string, trả bool  
Predicate<Product>    // nhận Product, trả bool
```

2. Ví dụ cơ bản

```
public static bool IsEven(int number)  
{  
    return number % 2 == 0;  
}  
  
Predicate<int> condition = IsEven;  
Console.WriteLine(condition(10)); // True
```

Vì IsEven nhận int và trả bool nên chữ ký khớp với Predicate<int>.

3. Vì sao dùng Find() mà không thấy chữ Predicate?

Khi gọi List<T>.Find(), chữ Predicate thường không xuất hiện ở dòng bạn viết. Nhưng nó nằm trong chữ ký của method Find(). Có thể hình dung Find được định nghĩa như sau:

```
public T? Find(Predicate<T> match)
```

Ví dụ:

```
List<int> numbers = new List<int> { 1, 3, 4, 7, 8 };  
int result = numbers.Find(IsEven);
```

Vì đây là List<int> nên T = int. Khi đó Find(Predicate<T>) trở thành Find(Predicate<int>). Compiler kiểm tra IsEven có dạng bool Method(int) hay không. Nếu khớp, method được truyền trực tiếp vào Find().

4. Viết đầy đủ để thấy Predicate rõ hơn

```
Predicate<int> condition = IsEven;  
int result = numbers.Find(condition);
```

C# cho phép bỏ biến trung gian và viết ngắn:

```
int result = numbers.Find(IsEven);
```

Đây là lý do bạn đang sử dụng Predicate nhưng có thể không nhìn thấy từ "Predicate" ở chỗ gọi.

5. Trường hợp List<string> và giá trị "SP01"

```
List<string> ids = new List<string>
{
    "SP01", "SP02", "SP03"
};

string? result = ids.Find(id => id == "SP01");
```

Ở đây List<string> nên T = string. Vì vậy Find cần Predicate<string>. Biểu thức điều kiện nhận một string và trả bool.

```
id => id == "SP01"

// tương đương về ý tưởng với:
public static bool IsSP01(string id)
{
    return id == "SP01";
}
```

6. Trường hợp List<Product>: điểm dễ nhầm nhất

```
public class Product
{
    public string Id { get; }
    public string Name { get; }

    public Product(string id, string name)
    {
        Id = id;
        Name = name;
    }
}

List<Product> products = new List<Product>
{
    new Product("SP01", "Mouse"),
    new Product("SP02", "Keyboard"),
    new Product("SP03", "Laptop")
};

Product? product = products.Find(p => p.Id == "SP01");
```

Dù bạn đang so sánh Id là string, Predicate ở đây vẫn là Predicate<Product>, không phải Predicate<string>. Lý do: Find() đang duyệt từng Product trong List<Product>.

```
p                // Product
p.Id              // string
"SP01"           // string dùng để so sánh
p.Id == "SP01"   // bool
```

Vì vậy toàn bộ điều kiện có dạng: Product → bool.

7. "SP01" không phải parameter của Predicate

Trong đoạn sau:

```
products.Find(product => product.Id == "SP01");
```

parameter của Predicate là product, vì mỗi lần Find kiểm tra một Product. "SP01" chỉ là giá trị dùng để so sánh với product.Id.

8. FindById(string id): phân biệt id và product

```
public Product? FindById(string id)
{
    return products.Find(product => product.Id == id);
}
```

Ở đây có hai thứ khác nhau:

- id là parameter kiểu string của method FindById().
- product là parameter của Lambda mà Find() sử dụng; kiểu của product là Product.

Luồng có thể hình dung:

```
FindById("SP01")
→ id = "SP01"
→ Find duyệt từng Product
→ kiểm tra product.Id == id
→ điều kiện trả true/false
```

9. Cách xác định Predicate<T> nhanh nhất

Đừng nhìn vào giá trị đang tìm. Hãy nhìn vào kiểu T của List<T>.

Danh sách	Predicate mà Find cần
List<string>	Predicate<string>
List<int>	Predicate<int>
List<Product>	Predicate<Product>
List<Student>	Predicate<Student>

Câu nhớ nhanh: T của Predicate<T> trong Find() chính là T của List<T>.

10. Find() thực chất làm gì?

Có thể tưởng tượng Find hoạt động gần giống đoạn code sau:

```
public static Product? FindProduct(
    List<Product> products,
    Predicate<Product> condition)
{
    foreach (Product product in products)
    {
        if (condition(product))
        {
            return product;
        }
    }
}
```

```
    return null;
}
```

Nếu condition đang trở tới IsExpensive thì condition(product) thực chất là IsExpensive(product).

11. Các method List<T> thường dùng Predicate

```
List<T>.Find(Predicate<T> match)
List<T>.FindAll(Predicate<T> match)
List<T>.Exists(Predicate<T> match)
List<T>.RemoveAll(Predicate<T> match)
```

- Find(): trả phần tử đầu tiên thỏa điều kiện.
- FindAll(): trả tất cả phần tử thỏa điều kiện.
- Exists(): kiểm tra có ít nhất một phần tử thỏa điều kiện hay không.
- RemoveAll(): xóa tất cả phần tử thỏa điều kiện.

12. Predicate<T> và Func<T, bool>

Về chữ ký, hai loại dưới đây rất giống nhau:

```
Predicate<Product>
Func<Product, bool>
```

Cả hai đều biểu diễn Product → bool. Tuy nhiên Predicate<T> nhấn mạnh ý nghĩa “đây là một điều kiện kiểm tra”.

13. Mức độ cần học ở giai đoạn hiện tại

Với Predicate<T>, hiện tại chỉ cần đạt các ý sau:

- Biết Predicate<T> nhận một T và luôn trả bool.
- Biết List<T>.Find(), FindAll(), Exists(), RemoveAll() có thể nhận Predicate<T>.
- Biết Predicate có thể nằm trong chữ ký API nên không nhất thiết xuất hiện ở dòng gọi.
- Biết với List<Product>, Find cần Predicate<Product>, dù điều kiện có thể kiểm tra property Id kiểu string.
- Biết Predicate<T> gần giống Func<T, bool> về chữ ký.

Không cần đào sâu hơn ở thời điểm này. Phần quan trọng tiếp theo là Lambda Expression.

14. Bài tập 9.5 - Predicate và List<T>.Find()

Cấu trúc đề xuất:

```
PredicateFindPractice/
├── Product.cs
├── ProductCondition.cs
├── ProductRepository.cs
└── Program.cs
```

Product.cs

```
public class Product
{
    public string Id { get; }
```

```

    public string Name { get; }
    public decimal Price { get; }

    public Product(string id, string name, decimal price)
    {
        Id = id;
        Name = name;
        Price = price;
    }

    public override string ToString()
    {
        return $"{Id} - {Name} - {Price}";
    }
}

```

ProductCondition.cs

Tạo hai method:

```

public static bool IsExpensive(Product product)
// Price >= 1000

public static bool IsCheap(Product product)
// Price < 100

```

ProductRepository.cs

Có field:

```

private readonly List<Product> _products;

```

Tạo method:

```

public Product? FindById(string id)

```

Yêu cầu: kiểm tra id null/empty/whitespace, Trim, dùng `_products.Find(...)` và so sánh Id không phân biệt hoa thường. Không dùng LINQ.

Program.cs

```

List<Product> products = new List<Product>
{
    new Product("SP01", "Mouse", 20),
    new Product("SP02", "Keyboard", 80),
    new Product("SP03", "Monitor", 300),
    new Product("SP04", "Laptop", 1500)
};

```

Thực hiện:

1. Tạo `Predicate<Product> condition = ProductCondition.IsExpensive`.
2. Dùng `condition` kiểm tra Product Laptop.
3. Dùng `products.Find(ProductCondition.IsExpensive)`.
4. Dùng `products.FindAll(ProductCondition.IsCheap)`.
5. Dùng `products.Exists(ProductCondition.IsExpensive)`.
6. Viết `FindById("SP02")`.
7. Test `FindById("sp02")` để kiểm tra không phân biệt hoa thường.

15. Câu hỏi cuối bài

8. Với `List<Product>`, tại sao `Find()` cần `Predicate<Product>` chứ không phải `Predicate<string>` dù ta đang tìm theo `Id` là `string`?
9. Trong `product => product.Id == "SP01"`, `product` có kiểu gì? Biểu thức toàn bộ trả về kiểu gì?
10. `"SP01"` có phải parameter của `Predicate` không? Vì sao?
11. Trong `FindById(string id)`, `id` và `product` trong `Lambda` khác nhau thế nào?
12. Vì sao `products.Find(ProductCondition.IsExpensive)` không cần viết rõ `Predicate<Product>` ở chỗ gọi?

16. Tóm tắt ghi nhớ

`Predicate<T>`

→ nhận 1 giá trị kiểu `T`

→ trả `bool`

`List<T>`

→ `Find` / `FindAll` / `Exists` / `RemoveAll` có thể nhận `Predicate<T>`

`List<Product>`

→ `Find` cần `Predicate<Product>`

`"SP01"` chỉ là giá trị so sánh; không quyết định kiểu `Predicate`.